

Summarizing Data

Location, variability, and outliers



Prof. Gabriel Spadon, PhD

Faculty of Computer Science
spadon@dal.ca

Announcements

- Prepare R or Python, locally or in a free cloud notebook.
- Cloud options include Posit Cloud, Kaggle Notebooks, and Google Colab (Python).
- Course-specific deadlines are published in the course GitHub repository.

Submit Privately, Then Share

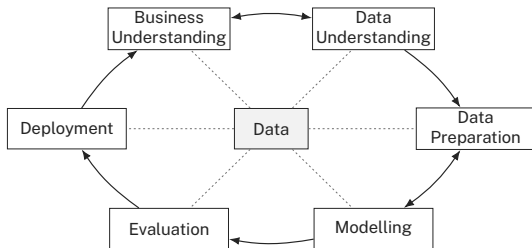
- Lab 01 is individual work, due **September 12 at 23:59 Halifax time**.
- The best 10 of 12 labs count for **15%**.
- Keep the repository and cloud notebooks private, with instructor access.
- Submit the full commit SHA and commit URL privately by the deadline.
- After the deadline and the instructor's release, publish that same commit and its history in your portfolio and present it in class.

No rewriting or backdating assessment history. Corrections use new commits.
Generative AI output is prohibited on labs.

Connect the Stages to a Question

1. Name Class A's six stages.
2. Which stage checks for missing temperatures?
3. If the units were wrong, which stage would you return to?

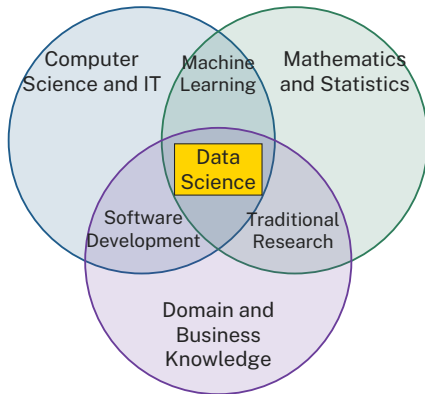
Checking measurements is data understanding; correcting their representation is data preparation.



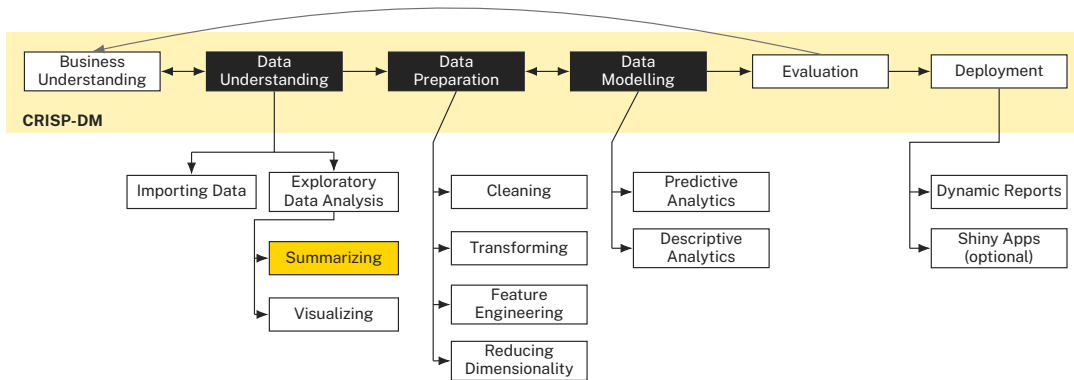
What Data Science is

Data science uses computing, statistics, and domain knowledge to learn from data and answer questions. Large datasets are not required.

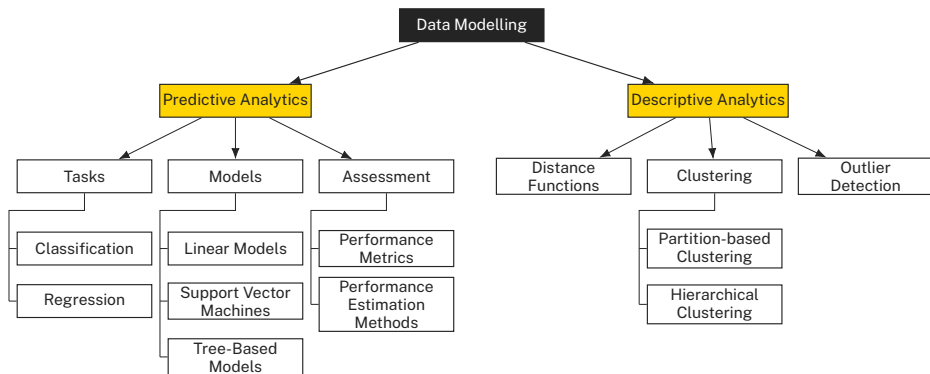
The overlap is the point. Computing makes analysis possible, statistics makes it defensible, and domain knowledge makes it useful. Adapted from Michael Barber.



Where Summarizing Sits



The Rest of the Map



Connections group topics; they do not prescribe an algorithm sequence.

Learning Objectives

LEARNING OBJECTIVES

- Calculate statistics of location and variability.
- Flag possible outliers using the boxplot rule.
- Generate different global dataset summaries.

AGENDA

- Location and spread, with hand calculations
- Outliers and missing values
- Guided R or Python practice, then Lab 01

Why Summarize Data?

- Data are often too large for us to be able to understand what is going on just by perusing them.
- Data summaries give us an overview of key data properties.
- We want to describe the **distribution** of the observed values.

2 431 rows × 18 columns



a handful of numbers



mean
median
sd, IQR
min, max

Types of Variables

■ Numerical

- Discrete, for example the number of visits
- Continuous, for example temperature

■ Categorical

- Nominal, such as blood group;
- ordinal, such as low, moderate, high

Storage types do not determine analytical roles.
Nominal data have no meaningful mean.

R

```
x <- c(2, 4, 4, 5, 7,  
      ↪ 9, 12, 41)  
class(x)  
str(x)
```

Python

```
import pandas as pd  
x = pd.Series([2, 4, 4, 5, 7, 9,  
              ↪ 12, 41])  
x.dtype
```

Three Questions to Ask

Q1. What is the “typical or common value” of a variable?

Q2. What is the “variability” in the values of a variable?

Q3. Are there unusual or unexpected values in the data?

—

These three questions organize the whole class, and each one has a small family of statistics attached to it.

Statistics of Location

Q1. What is the “typical or common value”?



Q1. The “Typical or Common Value”

Mean (or sample mean):

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- **Median:** at least half the observations lie at or below it, and at least half at or above it.
 - Sort the values and pick the middle value.
 - If there are an even number of values, calculate the mean of the two in the middle.
- **Mode:** most frequently occurring value.
 - The mode works with categorical variables.

WORKED EXAMPLE

$x = 2, 4, 4, 5, 7, 9, 12, 41$ ($n = 8$)

Mean. $\bar{x} = 84/8 = 10.5$.

Median. The two middle values are 5 and 7, so the median is $(5 + 7)/2 = 6$.

Mode. The value 4 occurs twice, so the mode is 4.

One observation of 41 pulls the mean far above the median.

Mean, Median, and Mode

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
mean(x)                # 10.5
median(x)               # 6
```

```
# ignore missing values
mean(c(2, NA, 4), na.rm = TRUE) # 3
```

```
# mode() reports storage, not frequency
tab <- table(x)
names(tab)[tab == max(tab)] # "4", all modes
```

Difference to know. A missing value is recorded as NA in R and NaN in a numeric pandas column. R needs `na.rm = TRUE` to omit it; pandas omits it by default. Always report the observed count. Both mode expressions return all tied modes.

Python

```
x = pd.Series([2,4,4,5,7,9,12,41])
x.mean()                # 10.5
x.median()               # 6.0
```

```
# pandas skips NaN by default
pd.Series([2, None, 4]).mean() # 3.0
```

```
x.mode().tolist()       # [4]
```

Getting an Accurate “Picture”

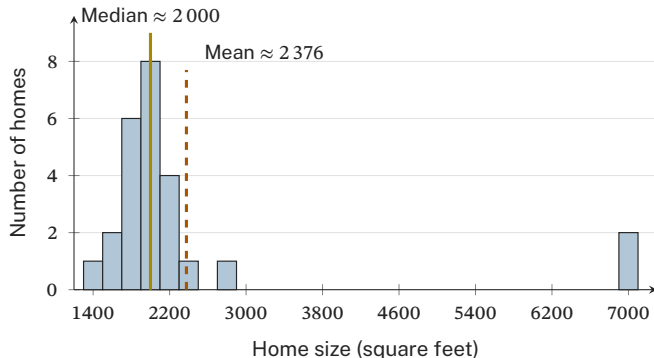
Ten people have an average income of \$50,000. Replace the highest earner with someone earning far more. The new mean can be arbitrarily large; the middle two incomes stay the same.

Hypothetical income example.

What happens to the median in this example?

The mean is not robust, because a single extreme value moves it without limit. The median is unchanged here because the fifth and sixth incomes are unchanged.

A Long Right Tail Moves the Mean

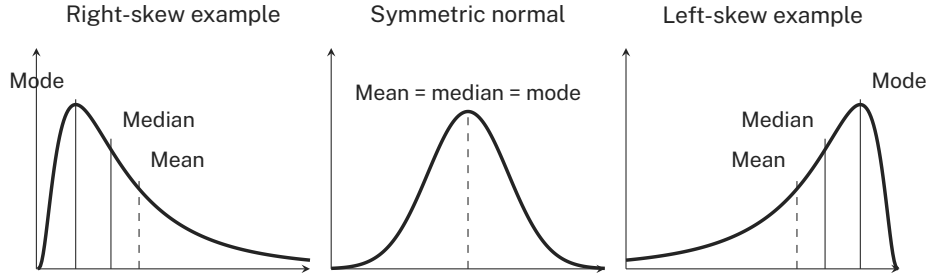


Twenty-five homes. Twenty-three of them lie between 1 400 and 2 800 square feet; two are near 7 000.

Using bin midpoints, the median is about 2 000 and the mean about 2 376. The distant homes pull the mean above the median.

The widening gap is the visual clue: a few large homes pull the mean away from the typical home. Binned reconstruction after [Statology](#).

Skewness Reorders the Statistics



These curves show the common ordering $\text{mode} < \text{median} < \text{mean}$ under right skew, with the reverse under left skew. This ordering is not universal. The normal example is symmetric.

Statistics of Variability or Spread

Q2. What is the “variability” in the values of a variable?



Q2. What is the “Variability”? I

Variance:

$$s_x^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Standard deviation:

$$s_x = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

What are the units of variance? What about standard deviation?

WORKED EXAMPLE

$x = 2, 4, 4, 5, 7, 9, 12, 41$, with $\bar{x} = 10.5$.

$$\sum_i (x_i - \bar{x})^2 = 1134$$

$$s_x^2 = 1134/7 = 162$$

$$s_x = \sqrt{162} = 12.73$$

Variance is in squared units of x ; the standard deviation returns to the units of x , which is why it is the one we usually report.

The divisor $n - 1$ makes this the *sample* variance s_x^2 , kept apart from the population variance σ_x^2 .

Variance and Standard Deviation

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
```

```
var(x) # 162      divisor n - 1  
sd(x)  # 12.72792
```

```
# Descriptive variance with divisor n  
var(x) * (length(x) - 1) / length(x)  
# 141.75
```

Python

```
x = pd.Series([2,4,4,5,7,9,12,41])
```

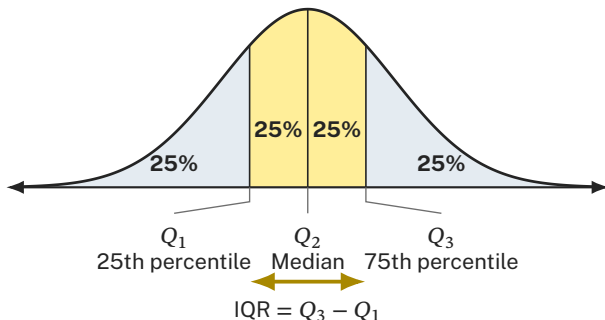
```
x.var()          # 162.0      ddof = 1  
x.std()          # 12.72792
```

```
x.var(ddof=0)    # 141.75, divisor n  
x.std(ddof=1)    # 12.72792
```

Difference to know. R's var/sd and pandas default to the sample divisor $n - 1$. In pandas, ddof=1 selects this divisor; ddof=0 selects n . Use the sample version for Lab 01.

Quartiles

- The p th percentile locates the lower $p\%$ of a distribution. Ties and sample interpolation affect the exact fraction.
- Quantiles mark chosen cumulative proportions.
- If data are separated into 4 parts, the quantiles are called *quartiles*:
 - the 1st quartile estimates the 25th percentile;
 - the 3rd quartile estimates the 75th percentile.



Normal example; both tails extend without bound.

For this continuous distribution, each region has probability 25%. A finite sample can contain ties. In R: `quantile(x, c(0.25, 0.75), type = 7)`. Adapted from Sufang [5].

Calculate a Type-7 Quartile

Sort the eight values 2, 4, 4, 5, 7, 9, 12, 41. For a proportion p , calculate the position $h = 1 + (n - 1)p$. Count positions from 1.

- For Q_1 , $p = 0.25$ and $h = 2.75$. Positions 2 and 3 both contain 4, so $Q_1 = 4$.
- For Q_3 , $p = 0.75$ and $h = 6.25$. Move one quarter of the way from position 6 to position 7.
- Those values are 9 and 12, so $Q_3 = 9 + 0.25(12 - 9) = 9.75$.

This calculation is linear interpolation. The following R and Python calls use this same definition.

Quantiles, and Which Definition

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)

quantile(x, c(0.25, 0.75), type = 7)
# 25% 75%
# 4.00 9.75      default type 7

fivenum(x)      # hinges, not type 7
# 2.0 4.0 6.0 10.5 41.0
```

Python

```
x = pd.Series([2,4,4,5,7,9,12,41])

x.quantile([0.25, 0.75],
            interpolation="linear")
# 0.25 4.00
# 0.75 9.75    linear == R type 7
```

Difference to know. Use R type 7 or pandas linear interpolation for Lab 01. R's `fivenum` and default `boxplot` use hinges, a different rule for dividing a sorted sample. Here their upper hinge is 10.5 rather than 9.75.

Q2. What is the “Variability”? II

INTERQUARTILE RANGE (IQR)

Difference between the 1st and 3rd quartile.

$$IQR = Q_3 - Q_1$$

For $x = 2, 4, 4, 5, 7, 9, 12, 41$: $Q_1 = 4$,
 $Q_3 = 9.75$, so $IQR = 5.75$.

RANGE

Difference between the maximum and minimum values.

$$\text{range} = x_{(n)} - x_{(1)}$$

For the same data: $41 - 2 = 39$, nearly seven times the IQR.

What do you think is an advantage of using the IQR as a measure of spread? Can you think of a disadvantage of using the range?

Outliers and Unknown Values

Q3. Are there any “strange” or unexpected values?



Q3. Are There Any “Strange” Values?

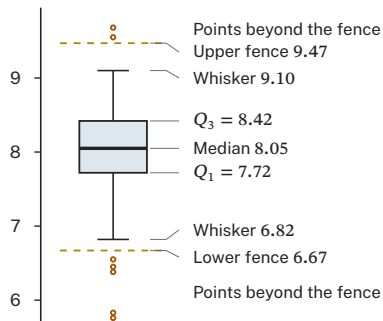
Informal definition. An outlier is an observation unusual enough to prompt investigation of its origin or measurement process.

Boxplot screening rule.

Flag observations outside this interval.

$$\left[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR \right]$$

A flagged observation is not automatically an error.



Illustrative values. Dashed lines are fences; whiskers stop at observations within them. Circles beyond the fences are flagged.

Applying the Boxplot Rule

$x = 2, 4, 4, 5, 7, 9, 12, 41$

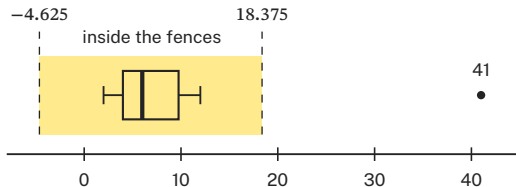
$Q_1 = 4, \quad Q_3 = 9.75, \quad IQR = 5.75$

$1.5 \times IQR = 8.625$

Lower fence: $4 - 8.625 = -4.625$

Upper fence: $9.75 + 8.625 = 18.375$

Only 41 lies outside $[-4.625, 18.375]$, so the rule flags exactly one value.



This diagram uses type-7 quartiles. Whiskers end at 2 and 12. R's default boxplot uses hinges, so its box and fences differ; both flag 41.

Missing and Obviously Wrong Values

- Missing or unknown values.

R

```
x <- c(2,4,4,5,7,9,12,41,NA)
```

- Other obviously wrong values, for example values of the wrong type.

```
boxplot.stats(x)$out  
# 41
```

```
which(is.na(x))  
# 9
```

```
sum(is.na(x))  
# 1
```

A flagged value is a question, not a verdict. Check the measurement process first.

Difference to know. `boxplot.stats` builds its fences from hinges, so they can differ slightly from fences built on quantile type 7. Here both flag only 41.

Python

```
x = pd.Series([2,4,4,5,7,9,12,41,None])
```

```
q1, q3 = x.quantile([.25, .75])  
iqr = q3 - q1  
x[(x < q1 - 1.5*iqr) |  
   (x > q3 + 1.5*iqr)]  
# 7    41.0
```

```
x.index[x.isna()]  
# Index([8])
```

Global Summaries in Two Languages

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
summary(x)
# Min. 1stQu. Median Mean
# 2.00 4.00 6.00 10.50
# 3rdQu. Max.
# 9.75 41.00
table(c("a", "b", "a", "c"))
# a b c
# 2 1 1
```

Difference to know. R's `summary()` reports the mean; pandas' `describe()` adds the standard deviation, computed with `ddof=1`, which matches R's `sd()`.

Python

```
import pandas as pd
x = pd.Series([2,4,4,5,7,9,12,41])
x.describe()
# count 8.00 min 2.00
# mean 10.50 25% 4.00
# std 12.73 50% 6.00
# 75% 9.75
# max 41.00
pd.Series(list("abac")).value_counts()
# a 2 / b 1 / c 1
x.agg(["mean", "median", "std"])
```

What to Take Away

Location

Mean, median, mode. Only the mode applies to nominal categories.

Variability

Variance, standard deviation, interquartile range, and range describe different aspects of spread.

Outliers

Flag possible outliers with the boxplot rule, then investigate their source and context.

Interpretation

Choose summaries for the question. Check plots, missingness, units, and software defaults.

Summary-statistics Warm-up

Practice in R or Python and generate basic data summaries.

Use a local or cloud environment. Compare your output with the worked example, then begin integrated Lab 01.

One Row Is One Sensor Reading

```
record_id,site,temperature_c,oxygen_percent,depth_m  
r01,A,2,70,1  
r02,A,4,80,2  
r03,A,4,80,2  
r04,A,5,85,3  
r05,B,7,90,3  
r06,B,9,95,4  
r07,B,12,105,4  
r08,B,41,130,10  
r09,A,,75,2
```

The blank field is missing, not zero. Lab 01 uses at least 100 observations from an approved source.

Save as `practice.csv`. These nine readings are invented for practice.

Temperature is in degrees Celsius, oxygen saturation in percent, and depth in metres. Site labels define groups; record IDs identify rows.

Read the CSV and Select Measurements

R

```
observations <- read.csv("practice.csv",  
  ↪ stringsAsFactors = FALSE)  
numeric_cols <- c("temperature_c", "  
  ↪ oxygen_percent", "depth_m")  
dir.create("results", showWarnings = FALSE)
```

Python

```
from pathlib import Path  
  
import matplotlib.pyplot as plt  
import pandas as pd  
  
observations = pd.read_csv("practice.csv")  
numeric_cols = ["temperature_c", "  
  ↪ oxygen_percent", "depth_m"]  
Path("results").mkdir(exist_ok=True)
```

Put the script and CSV in the same working folder. The complete example in the notes loops through these three columns and writes one row of summaries for each. Follow that example during the demonstration.

Check the Output Before Adapting It

Overall temperature

- 8 observed, 1 missing
- Mean 10.5, median 6
- Sample SD 12.73
- $Q_1 = 4$, $Q_3 = 9.75$, IQR 5.75
- Type-7 fences flag 41 (row r08)

Compare the sites

- A has 4 observed temperatures, mean 3.75
- B has 4 observed temperatures, mean 17.25
- The value 41 contributes to the difference
- A statistical flag alone cannot establish a measurement error

Save Evidence That Can Be Checked

1. Save the summary, group comparison, and flagged rows as CSV tables.
2. Make a temperature histogram and a temperature–oxygen scatter plot. Label variables, units, and the number of observations used.
3. Save the plots as PDFs. Record the software environment and the command that runs the script.
4. Restart and run the full analysis. Check that it recreates every result cited in the report.

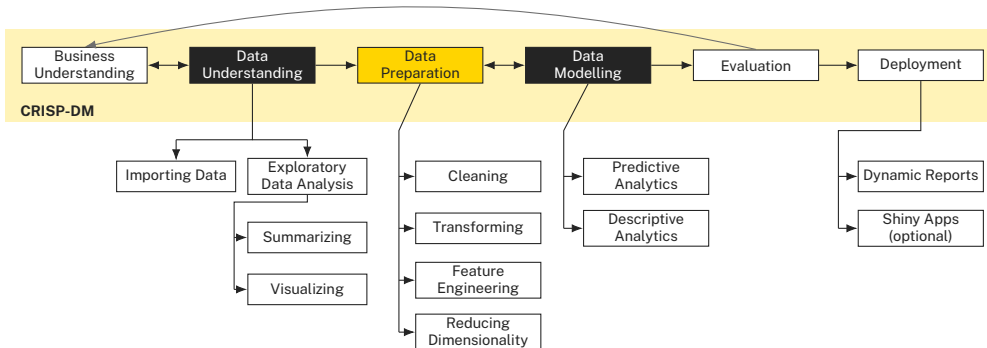
For Lab 01, also record the dataset publisher, URL, licence, access date, selected station and period, and missing-value codes. Keep the original download unchanged.

Commit, Check, and Submit Privately

1. Create a private GitHub repository and give the instructor access. Store the lab under `labs/lab-01/`.
2. A commit records a snapshot. Use the browser's *Commit changes*, or commit and push from local Git. Record meaningful stages as you work.
3. Open the final commit on GitHub and check its files. Submit its full SHA and commit URL privately by September 12 at 23:59 Halifax time.

The notes give the steps and commands. Corrections are new commits. Publish the same submitted commit and history only after the deadline and the instructor's release.

Next Lecture: Data Preparation



The same map advances to data preparation. The next class begins preprocessing foundations; later classes develop the remaining preparation topics.

Lab 01 | Atlantic environmental conditions

Describe one open Atlantic environmental dataset.

Carry forward

Observational units, variable types, centre, spread, quantiles, missingness, and unusual-value investigation.

Start in class

Choose one verified dataset. Work through the selection checklist, then set up the R or Python analysis.

One assessment, two languages

Submit either R or Python.
Both paths use the same evidence and rubric.

`week-01/Assessment/lab-01`

Why Check a Model's Input?

The required Week 1 work ends with Lab 01. The following research examples are optional and are not needed for the lab.

- A classifier predicts a category, as the tree did in Class A.
- Training images are examples used to fit it. Test images are separate examples used to examine its predictions.
- A perturbation is a change to an input. Retraining means fitting the model again after changing its training data.
- Confidence is the score the model assigns to its prediction. A large score can accompany a mistake.

Background Shortcuts

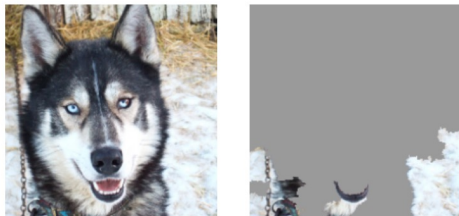
An illustrative warning



These different scenes illustrate a possible association between the background and the label. The tank story is apocryphal; the pictures do not establish a measured model failure.

Guidotti et al., Figure 1 [1].

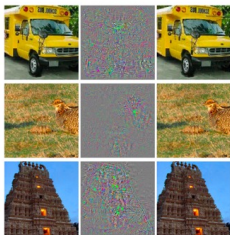
A documented shortcut



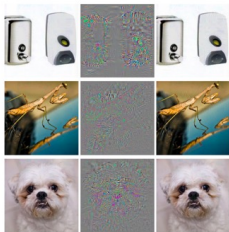
A husky predicted as *wolf* (left), with an explanation focused mainly on snow (right). Both panels refer to the same input.

Ribeiro et al., Figure 11 [4].

Change the Test Input



(a)



(b)

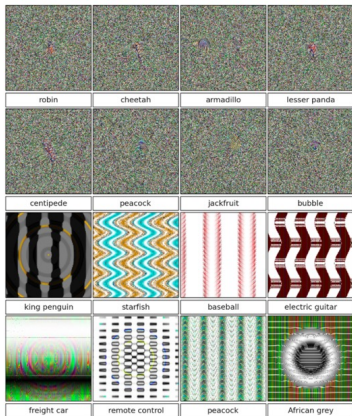
Each triplet shows the original, a magnified difference, and the altered image.

The trained model stays fixed.

The middle column magnifies the perturbation by 10× for visibility. The altered images still look similar to the originals, but the image classifier AlexNet predicts *ostrich* for each right-hand image.

Szegedy et al., Figure 5 [6].

High Confidence Without Recognition



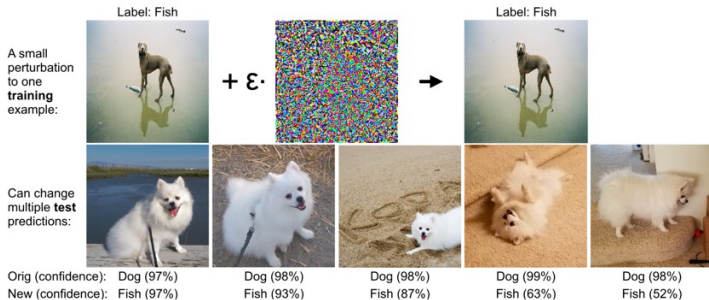
A class label can receive high confidence even when a person sees no recognizable object.

These are the published optimized images. The labels underneath are the network's predictions.

The top half contains noise-like patterns; the bottom half contains structured patterns. Nguyen et al. report at least 99.6% confidence for the examples in this figure. That confidence is a property of the model, not proof that the object is present.

Nguyen et al., Figure 1 [3].

Change Training Data, Then Retrain



Top, one training image is perturbed. Bottom, separate test images receive changed predictions after retraining.

The test images stay fixed.

The training label remains *fish*. The perturbed training example affects the fitted model. In this experiment, predictions changed from dog to fish for 16 of 30 targeted test images.

Koh and Liang, Figure 5 [2].

References

- [1] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi. “A survey of methods for explaining black box models”. In: *ACM Computing Surveys* 51.5 (2018), pp. 1–42.
- [2] P. W. Koh and P. Liang. “Understanding black-box predictions via influence functions”. In: *Proceedings of the 34th International Conference on Machine Learning*. PMLR, 2017.
- [3] A. Nguyen, J. Yosinski, and J. Clune. “Deep neural networks are easily fooled: high confidence predictions for unrecognizable images”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2015.
- [4] M. T. Ribeiro, S. Singh, and C. Guestrin. ““Why should I trust you?” Explaining the predictions of any classifier”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016.
- [5] W. Sufang. “An adaptive ensemble classification framework for real-time data streams by distributed control systems”. In: *Neural Computing and Applications* 32.9 (2020), pp. 4139–4149.
- [6] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. *Intriguing properties of neural networks*. arXiv preprint arXiv:1312.6199. 2013.